

UNIVERSIDAD EAFIT
ANÁLISIS DE ALGORITMOS
PRÁCTICA FINAL (INTEGRADORA) 2026/01 (15%)

Tema: Optimización de rutas y planificación de recursos en redes de telecomunicaciones – Algoritmos Codiciosos, Divide y Vencerás, y Programación Dinámica sobre datos reales

Modalidad: Actividad grupal, máximo 3 integrantes.

Fecha de entrega: viernes 08 de mayo de 2026, hasta las 11:59 p.m.

Medio de entrega: Enlace al repositorio GitHub publicado en InteractivaVirtual, con los nombres completos de los integrantes del equipo.

Lenguaje: C++ (implementación obligatoria).

1. Contexto y motivación

Un operador que despliega fibra óptica sobre una ciudad real debe resolver simultáneamente tres subproblemas: ordenar y consultar eficientemente su inventario de solicitudes de servicio (Divide y Vencerás), tender el menor cableado posible para interconectar sus nodos de red (Codicioso), y asignar el ancho de banda disponible entre solicitudes competidoras para maximizar el ingreso (Programación Dinámica). Este es un problema práctico de planificación de infraestructura de telecomunicaciones.

Este proyecto utiliza el dataset Telecom Customer Churn disponible en Kaggle, enriquecido con un grafo sintético de topología de red generado de forma determinista a partir de los datos, de modo que los resultados sean reproducibles y verificables entre equipos. El dataset contiene registros de clientes con atributos de consumo (minutos de uso, cargos, región) que se interpretan como solicitudes de servicio a procesar, ordenar y asignar sobre la infraestructura.

Fuente del dataset:

- **URL:** <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- **Registros:** 7,043 clientes, 21 atributos.
- **Licencia:** CC0 (dominio público). No requiere cuenta Kaggle para descarga directa desde el repositorio del curso.
- El archivo WA_Fn-UseC_-Telco-Customer-Churn.csv debe incluirse en el repositorio bajo data/.

La topología de red se construye a partir del dataset mediante el procedimiento descrito en la Sección 9 (Lineamientos técnicos), garantizando que todos los equipos operen sobre el mismo grafo con los mismos pesos.

2. Objetivos

- Aplicar Divide y Vencerás para ordenar y consultar eficientemente un conjunto de solicitudes de servicio extraídas de un dataset real.
- Implementar un Algoritmo Codicioso (Kruskal) para construir la red de mínimo costo que interconecte los nodos de infraestructura derivados del dataset.

- Utilizar Programación Dinámica (Mochila 0-1 por tabulación) para asignar ancho de banda limitado entre solicitudes, y demostrar formalmente el fallo del enfoque codicioso en este subproblema.
- Integrar los tres módulos en un pipeline coherente donde la salida de un módulo alimenta la entrada del siguiente.

3. Descripción del problema y tareas requeridas

Módulo A – Procesamiento de solicitudes de servicio (Divide y Vencerás)

A partir del archivo CSV, cada registro de cliente se interpreta como una solicitud de servicio con los siguientes campos extraídos:

Campo en CSV	Interpretación en el problema
customerID	Identificador de solicitud
tenure	Prioridad de servicio (meses de antigüedad)
MonthlyCharges	Valor mensual de la solicitud (en USD)
TotalCharges	Valor acumulado (peso de la solicitud en la mochila)
Churn	Estado: solicitud activa (No) o en riesgo (Yes)

Actividades:

- 1. Parseo del CSV:** Leer el archivo ignorando la cabecera. Manejar el campo TotalCharges que contiene espacios en blanco en registros con tenure = 0; asignar valor 0.0 en esos casos. Reportar el número total de registros cargados y el número de registros con TotalCharges nulo.
- 2. Ordenamiento por MergeSort:** Implementar MergeSort para ordenar las solicitudes por tenure descendente (mayor prioridad primero). Justificar en el informe por qué MergeSort es preferible a QuickSort cuando se requiere estabilidad y comportamiento garantizado en $O(n \log n)$ en el peor caso.
- 3. Búsqueda binaria recursiva:** Sobre el arreglo ordenado, implementar una función de búsqueda binaria recursiva que localice la primera solicitud cuyo tenure sea mayor o igual a un valor de consulta k. Ejecutar las siguientes 5 consultas fijas y reportar el customerID del resultado:

Consulta	Valor de k
Q_A01	72
Q_A02	60
Q_A03	45
Q_A04	30
Q_A05	12

- 4. Análisis empírico:** Medir el tiempo de ejecución del ordenamiento para los 7,043 registros completos y para submuestras de 1,000 y 3,500 registros. Graficar (tabla) y verificar que la relación entre tiempos es consistente con $O(n \log n)$.

Salida: results/solicitudes_ordenadas.csv con los registros ordenados, y results/busquedas_A.txt con los resultados de las 5 consultas.

Módulo B – Red de mínimo costo (Codicioso / Kruskal)

A partir de los datos del dataset se construye un grafo de infraestructura de red siguiendo el procedimiento determinista de la Sección 9. El grafo representa nodos de concentración (agrupaciones de clientes por región sintética) y aristas con costos de instalación derivados de los cargos promedio del grupo.

Actividades:

- 1. Construcción del grafo:** Seguir el procedimiento de la Sección 9 para generar el grafo de $N = 20$ nodos y sus aristas ponderadas. Reportar el número de nodos, número de aristas y el costo promedio de arista.
- 2. MST por Kruskal:** Implementar el algoritmo de Kruskal con Union-Find (unión por rango y compresión de caminos). Reportar el peso total del MST y la lista de aristas incluidas.
- 3. Justificación de la propiedad de elección codiciosa:** Demostrar en el informe, usando el Lema del ciclo, por qué seleccionar la arista de menor peso que no forme ciclo garantiza el óptimo global. Ilustrar con una arista concreta del grafo generado.
- 4. Verificación:** Calcular manualmente el MST para los primeros 5 nodos del grafo (subgrafo inducido) y verificar que Kruskal produce el mismo resultado.

Salida: results/mst_red.txt con la lista de aristas del MST, el peso total, y el reporte de la propiedad de elección codiciosa.

Módulo C – Asignación de ancho de banda (Programación Dinámica)

Un nodo concentrador tiene una capacidad de ancho de banda de $W = 500$ unidades. Se toman las 50 solicitudes activas de mayor tenure (las primeras 50 del arreglo ordenado en el Módulo A con Churn = No). Para cada solicitud i :

- **Peso w_i :** TotalCharges redondeado al entero más cercano (unidades de ancho de banda requeridas).
- **Valor v_i :** MonthlyCharges multiplicado por 10 y redondeado (ingreso mensual en centavos).

Actividades:

- 1. Mochila 0-1 por tabulación:** Construir la tabla $dp[i][w]$ de dimensiones $(51) \times (W + 1)$ e implementar la solución completa. Reportar el valor óptimo total, el número de solicitudes seleccionadas, y listar los customerID incluidos.
- 2. Fallo del enfoque codicioso:** Construir un contraejemplo explícito usando exactamente 3 solicitudes del conjunto anterior donde el codicioso por ratio v_i/w_i no produce el óptimo. Presentar la tabla comparativa:

Enfoque	Solicitudes seleccionadas	Valor total	¿Óptimo?
Codicioso (ratio v/w)	...	...	No
PD (Mochila 0-1)	...	...	Sí

- 3. Reconstrucción de la solución:** Implementar el backtracking sobre la tabla dp para recuperar el conjunto exacto de solicitudes seleccionadas.

- 4. Análisis de complejidad:** Justificar formalmente que la complejidad es $\Theta(n \cdot W)$ en tiempo y espacio, y discutir si este algoritmo es polinomial en sentido estricto (pseudopolinomialidad).

Salida: results/asignacion_bw.txt con la tabla de la solución óptima, el contraejemplo codicioso y el análisis de complejidad.

4. Integración del pipeline

Los tres módulos no son independientes: el Módulo A produce el ordenamiento que alimenta la selección de solicitudes del Módulo C; el grafo del Módulo B se construye a partir de las regiones sintéticas derivadas del mismo dataset. El programa principal debe ejecutarse con:

```
bash
g++ -std=c++17 -O2 -o ada_pf src/main.cpp src/*.cpp
./ada_pf data/WA_Fn-UseC_-Telco-Customer-Churn.csv
```

Y producir todos los archivos de results/ en una sola ejecución.

5. Estructura del repositorio en GitHub

El nombre del repositorio debe seguir el formato: ADA_PF_Apellido1_Apellido2_Apellido3.

```
ADA_PF_Apellido1_Apellido2_Apellido3/
├── README.md                Instrucciones completas de compilación y ejecución
├── data/
│   └── WA_Fn-UseC_-Telco-Customer-Churn.csv  Dataset (incluir en el repositorio)
├── results/
│   ├── solicitudes_ordenadas.csv             Módulo A: registros ordenados por tenure
│   ├── busquedas_A.txt                      Módulo A: resultados de las 5 consultas
│   ├── mst_red.txt                          Módulo B: aristas del MST y peso total
│   └── asignacion_bw.txt                    Módulo C: solución óptima y contraejemplo
├── src/
│   ├── parser.cpp / parser.hpp              Lectura y parseo del CSV
│   ├── mergesort.cpp / mergesort.hpp
│   ├── binary_search.cpp / binary_search.hpp
│   ├── graph.cpp / graph.hpp                Grafo ponderado y Union-Find
│   ├── kruskal.cpp / kruskal.hpp
│   ├── knapsack.cpp / knapsack.hpp
│   └── main.cpp                             Orquestador del pipeline completo
├── report/
│   └── Informe.pdf
```

6. Informe técnico (report/Informe.pdf)

Documento técnico conciso, máximo 10 páginas sin contar portada, con los siguientes apartados:

- 1. Introducción:** Descripción del problema, contexto del dataset y justificación de por qué cada módulo requiere su paradigma específico.
- 2. Descripción del dataset:** Estadísticas del CSV cargado; manejo de valores nulos en TotalCharges; descripción del procedimiento de construcción del grafo.
- 3. Módulo A – Divide y Vencerás:** Pseudocódigo de MergeSort y búsqueda binaria; análisis de complejidad; tabla de tiempos empíricos para los tres tamaños de muestra; resultados de las 5 consultas.
- 4. Módulo B – Codicioso:** Pseudocódigo de Kruskal con Union-Find; demostración del Lema del ciclo aplicado al grafo generado; lista de aristas del MST y peso total.
- 5. Módulo C – Programación Dinámica:** Formulación de la recurrencia de la Mochila 0-1; pseudocódigo de la tabulación y el backtracking; contraejemplo codicioso con tabla comparativa; discusión de pseudopolinomialidad.
- 6. Integración del pipeline:** Diagrama de flujo del programa completo; cómo la salida del Módulo A alimenta al Módulo C.

7. Conclusiones: Comparación de los tres paradigmas en términos de garantías de optimalidad y complejidad; reflexión sobre el comportamiento en datos reales frente a instancias sintéticas.

8. Herramientas utilizadas: Citación explícita de cualquier herramienta de IA generativa usada (ver Sección 7).

9. Referencias: Dataset de Kaggle, material del curso, bibliografía consultada.

10. Roles del equipo: Contribución específica de cada integrante por módulo.

7. Rúbrica de evaluación

Criterio	Excelente 4.6–5.0	Bueno 4.1–4.5	Satisfactorio o 3.6–4.0	Insuficiente e 3.0–3.5	Deficiente 0.1–2.9	No realizado 0
Módulo A (D&V)	MergeSort correcto y estable; búsqueda binaria recursiva correcta; 5 consultas con resultado exacto; tiempos empíricos consistentes con $O(n \log n)$.	Correcto con análisis empírico incompleto o 1 consulta fallida.	Ordenamiento correcto, pero búsqueda iterativa (no recursiva) o sin análisis empírico.	Solo ordenamiento funcional; búsqueda ausente o incorrecta.	Ordenamiento con errores sistemáticos.	No implementado.
Módulo B (Greedy)	Kruskal con Union-Find correcto; MST verificado manualmente en subgrafo de 5 nodos; demostración del Lema del ciclo con arista concreta.	Correcto sin verificación manual o con demostración incompleta.	MST correcto sin Union-Find optimizado o sin demostración teórica.	MST incorrecto, pero estructura presente.	Algoritmo sin Union-Find y con errores en el MST.	No implementado.
Módulo C (PD)	Tabulación correcta; backtracking implementado; contraejemplo codicioso explícito con tabla comparativa; discusión de pseudopolinomialidad.	Correcto sin backtracking o sin discusión de pseudopolinomialidad.	Tabulación correcta, pero sin contraejemplo o sin reconstrucción.	Tabla dp construida con errores; sin backtracking ni contraejemplo.	Intento sin resultados válidos.	No implementado.
Pipeline e integración	Ejecución en un solo comando; Módulo A alimenta Módulo C correctamente; todos los archivos de results/ generados.	Un archivo de salida faltante o integración parcial.	Módulos funcionales, pero sin integración en pipeline; ejecución manual por separado.	Solo 1 módulo integrado al main.	Código no compila.	Sin código.
Informe y calidad técnica	Todas las secciones presentes; pseudocódigos correctos; análisis de complejidad formal; README reproducible.	Informe adecuado con carencias en 1–2 secciones.	Informe superficial o sin análisis formal de complejidad.	Informe muy incompleto; README ausente.	Informe irrelevante; código sin documentación.	Sin informe.

8. Uso ético de herramientas de Inteligencia Artificial

Usos permitidos (con citación obligatoria en la Sección 8 del informe):

- Consultar conceptos teóricos sobre los paradigmas o sobre la Mochila 0-1.
- Solicitar aclaraciones sobre errores de compilación o comportamiento inesperado del código.
- Obtener retroalimentación sobre la redacción del informe.
- Explorar variantes de implementación para comprender diferencias conceptuales.

Usos prohibidos (constituyen plagio académico):

- Solicitar a una IA que genere el código fuente de cualquiera de los módulos y entregarlo sin comprenderlo.
- Solicitar que la IA procese el dataset o genere los archivos de resultados.
- Copiar el análisis de complejidad o el contraejemplo codicioso directamente desde una respuesta de IA sin reescritura y comprensión propia.
- Usar la IA para eludir el aprendizaje: si un integrante no puede explicar su propio código en una revisión oral, la entrega se considera inválida.

El incumplimiento se penaliza con nota 0.0 en la actividad y reporte académico formal ante la Dirección de Programa.

9. Lineamientos técnicos

Parseo del CSV:

- Ignorar la primera línea (cabecera).
- El campo TotalCharges contiene un espacio en blanco (no un número) para los 11 clientes con tenure = 0; tratar como 0.0.
- Extraer únicamente los campos customerID, tenure, MonthlyCharges, TotalCharges y Churn.

Construcción determinista del grafo para el Módulo B:

- Agrupar los 7,043 registros en $N = 20$ grupos numerados $0, 1, \dots, 19$ asignando el registro i al grupo $i \bmod 20$.
- El costo de la arista entre los nodos u y v (con $u < v$) se define como:

$$c(u, v) = \lfloor \overline{M_u} + \overline{M_v} \rfloor$$

donde $\overline{M_k}$ es el promedio de MonthlyCharges de todos los registros del grupo k , redondeado a dos decimales antes de la suma.

- Crear aristas para todos los pares (u, v) con $u < v$, resultando en un grafo completo de 20 nodos y 190 aristas. Todos los equipos deben obtener exactamente el mismo grafo si siguen este procedimiento.

Medición de tiempos:

- Usar `std::chrono::high_resolution_clock`.
- Reportar en milisegundos con dos decimales.

Compilación:

```
bash
g++ -std=c++17 -O2 -o ada_pf src/main.cpp src/*.cpp
```

10. Verificación de resultados

Los equipos pueden verificar sus resultados del Módulo A contra estos valores de referencia derivados del dataset original:

Propiedad	Valor de referencia
Total de registros cargados	7,043
Registros con TotalCharges nulo	11
Registros con Churn = No	5,174
tenure máximo en el dataset	72
tenure mínimo en el dataset	0
MonthlyCharges promedio global	64.76 USD

Los resultados del Módulo B (MST) y el Módulo C (Mochila) no tienen valor único publicado externamente, pero todos los equipos que sigan los procedimientos de la Sección 9 deben obtener exactamente los mismos valores, lo que permite verificación cruzada entre equipos.

11. Conexión con contenidos del curso

Contenido del curso	Aplicación en el proyecto
Divide y Vencerás: MergeSort	Módulo A: ordenamiento estable de 7,043 solicitudes
Divide y Vencerás: búsqueda binaria	Módulo A: consultas en $O(\log n)$ sobre datos ordenados
Teorema Maestro	Informe: justificación de $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$
Algoritmos Codiciosos: propiedades de optimalidad	Módulo B: Lema del ciclo en MST
Kruskal y Union-Find	Módulo B: construcción eficiente del MST
Programación Dinámica: subestructura óptima	Módulo C: formulación de la recurrencia de la Mochila 0-1
Fallo de la estrategia codiciosa	Módulo C: contraejemplo explícito con datos reales
Análisis de complejidad	Informe: justificación formal de $O(n \log n)$, $O(E \log E)$ y $\Theta(n \cdot W)$